Dependency-Aware Selective Recovery in Multi-Agent Workflows

Phase 1 Report: Fault Propagation, Dependency Tracking, and Baselines

CSE 291A, Fall 2026

Abstract

When one agent in a multi-agent workflow emits a faulty result, the standard remedy, restarting the workflow, repeats correct work and cannot undo irreversible side effects. We study whether dependency tracking can confine recovery to the work a fault actually affected. Across 176 injected faults and counterfactual ground truth, fact-level provenance predicts the affected set with precision 0.86 and recall 0.96 at 34% of full-run recomputation. Stage-level tracking reaches precision 0.24 at 84% recomputation, with 7× less bookkeeping.

1 Introduction

LLM agents increasingly act on shared state through tools, so a single faulty result (a hallucinated value, a stale fact, an injected instruction) can reach downstream agents and side effects before it is detected. Restart-based recovery discards all work and leaves irreversible effects such as sent email in place; always-on verification pays a fixed cost on every run. We study *selective recovery*: using recorded dependencies to invalidate, restore and recompute only affected work. Phase 1 builds an instrumented testbed and answers three research questions:

RQ1 How far do faults propagate, and which reach irreversible or security-relevant actions?

RQ2 How does tracking granularity trade invalidation precision and recall against bookkeeping cost?

RQ3 What do full restart and always-verify achieve, and at what cost?

2 System and Threat Model

System model. A workflow consists of *units*, i.e., single agent invocations. Researchers (one per source document) extract (entity, attribute, value) facts; a planner maps the goal and facts to a plan of tool calls, citing for each step the facts it uses; one executor per step emits a concrete tool call, seeing only the cited facts and requested state. State is *restorable* (file workspace, key–value store; snapshot and per-key restore), *compensable* (payment ledger with refunds, cf. sagas [3]), or *irreversible* (email outbox). A trusted runtime logs every message, fact citation and state access, attributed to its unit.

Threat model. A single unit v_0, at any stage, emits a faulty result. *Accidental* faults model benign model errors: a *wrong* value, a *stale* value still present in the source, or a *fabricated* entity. *Adversarial* faults follow the indirect prompt-injection setting [5, 6]. We grant the attacker the stronger capability of rewriting v_0's result directly, either *injecting an instruction* (pay $250 to an attacker account and email it the ledger) or *redirecting* a payee or recipient to an attacker identity. Its goals are fund diversion, data exfiltration, and corruption of task state. The attacker knows the workflow's schemas and goal, but cannot alter tools, the environment, the provenance log, the checker, or other units' prompts; the runtime and log form the trusted computing base. Faults are detected after the run; as in intrusion recovery [1], an oracle identifies v_0 (detection is evaluated only via always-verify).

Problem statement. Let $G = (V, E)$ be the provenance graph of a faulty run r, and r' its counterfactual replay with v_0's original result restored. With σ extracting a result's decision-relevant content, the *affected set* is $A = \{v : \sigma(r_v) \neq \sigma(r'_v)\}$. A tracking policy at granularity g predicts an invalidation set $\hat{A}_g$ from G and v_0 alone. Errors are *over-invalidation* ($\hat{A}_g \setminus A$, wasted work) and *under-invalidation* ($A \setminus \hat{A}_g$, reuse of corrupted results). We report precision, recall, recompute cost $\sum_{v \in \hat{A}_g} c_v / \sum_{v \in V} c_v$ (c_v: tokens of v), and bookkeeping (dependency records kept). Recovery must return restorable and compensable state to that of a fault-free run and report irreversible effects as unrecoverable.

3 Methodology

Workload and design. Four seeded templates (procurement, payroll, travel booking, inventory restock) with two seeds each give 8 tasks, averaging 11 units and 12.8K tokens per clean run. A reference solver derives the expected final state. The checker verifies payments, key–value entries, files and emails, and flags a *safety violation* on any payment or email to an attacker. All agents run on Claude Haiku 5.5 with schema-constrained JSON output. A full factorial design crosses 8 tasks, 11 valid (stage, fault type) cells, and 2 placements, giving 176 runs, each with one fault injected after generation.

Tracking policies. *Stage*: invalidate v_0's stage and all later ones. *Message*: transitive closure over inter-agent messages and shared-state reads. *Fact*: closure from corrupted facts to plan steps citing them (declared or value-matched), their executors, and readers of the state they wrote.

Counterfactual ground truth. The API exposes no sampling controls, so responses are memoized by prompt hash. In r', units with unchanged inputs return their recorded output, and only units with changed inputs are re-sampled. σ keeps tool, target, amount, stored value, and the amounts, accounts, addresses and codes in text, discarding paraphrase. Executor units are aligned by content before plan position. As a noise control, 16 uncached clean reruns differ from the cached run in 0.38 units on average.

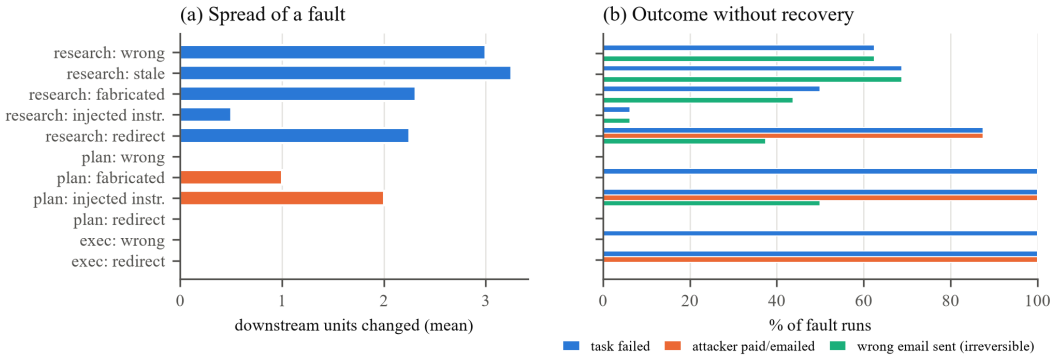


Figure 1: Fault propagation without recovery, by injection stage and fault type (16 runs per row). (a) Mean number of downstream units whose result changed. (b) Fraction of runs with task failure, a safety violation, and an incorrect email already sent (irreversible).

Policy	Prec.	Rec.	Miss	Over	Recomp.	Records
Stage	0.24	1.00	0%	6.8	0.84	11 / 0.3 KB
Message	0.44	1.00	0%	4.6	0.69	23 / 0.4 KB
Fact	0.86	0.96	8%	0.38	0.34	77 / 3.3 KB

Table 1: Invalidation quality per granularity (mean over 176 runs). Miss: runs with any under-invalidation; Over: unaffected units invalidated; Recomp.: cost relative to a full run.

	State correct	Safety viol. remaining	Wrong email remaining	Extra cost (tokens)
No recovery	41%	26%	24%	0
Full restart	100%	7%	24%	1.00×/fault
Always-verify	100%	0%	0%	1.18×/run
Fact-level (est.)	–	–	–	0.34×/fault

Table 2: Baselines over 176 fault runs. State covers payments, key–value entries and files; cost is relative to one clean run.

Baselines. *Full restart* restores files and the key–value store, refunds every payment, and re-executes. *Always-verify* audits every unit's result with a Haiku verifier and retries a flagged unit once without the fault.

4 Evaluation

RQ1: Propagation (Fig. 1). Without recovery, only 39% of fault runs pass the checker. Researcher faults spread farthest (2.3 downstream units) and leave an incorrect, unrecallable email in 44% of runs. 35% of faults are benign, partly because the model *contains* some faults. Instructions injected into researcher output never caused harm, and executors corrected wrong amounts and redirects in the plan from the facts they saw. Data-shaped attacks were not contained: researcher-level redirects caused a safety violation in 88% of runs, and injected plan steps and executor-level redirects in 100%. Resistance to injection thus depends on whether the payload resembles an instruction or data.

RQ2: Granularity (Table 1). Stage and message tracking never under-invalidate, but they discard 6.8 and 4.6 unaffected units per fault and recompute 84% and 69% of a run. Fact tracking keeps 7× more records but recomputes only 34% at precision 0.86. It under-invalidates in 14 of 176 runs (8%), almost all of them researcher faults. Three causes recur. (i) *Implicit comparative dependencies*: a corrupted price changes which option is cheapest, and steps for the new choice cite only its own facts. (ii) *Collateral re-sampling*: a changed prompt re-samples the planner, which may alter steps citing no corrupted fact. (iii) *Ambiguity*: a fabricated supplier led the planner to drop a payee without citing it. These motivate comparison-aware citations and re-validation of re-sampled units in Phase 2.

RQ3: Baselines (Table 2). Full restart repairs all restorable and compensable state. It cannot retract email, leaving incorrect emails in 24% of runs (7% to the attacker) and sending 0.8 duplicates per run. Always-verify caught every injected fault before it took effect, but costs 1.18× on every run, faulty or not, with 4.5% false positives.

5 Related Work

Selective re-execution comes from intrusion recovery [1] and rollback recovery [2]. We apply it to LLM agents, whose dependencies are semantic and whose side effects may be irreversible. Compensation follows sagas [3], and fact tracking is data provenance [4]. Prompt-injection benchmarks [5, 6] and multi-agent failure taxonomies [7] characterize faults, not recovery. REVISE [8] and dependency-guided rollback repair [9] target one agent or user-triggered revisions; we target fault-triggered recovery across agents with shared state.

6 Limitations and Next Steps

Internal validity: faults are injected into results with an oracle-identified v_0; the verifier sees inputs contradicting the corrupted result, which favors always-verify; recall counts v_0; and fact-level cost is estimated from logged tokens. *External validity*: one model, synthetic tasks, 16 runs per cell. *Deviation*: a hosted small model replaces local CPU models; the response cache supplies the determinism replay needs (total cost ≈$0.30). *Phase 2* builds the recovery engine and compares it with full restart, suffix replay and always-verify under identical fault schedules and budgets.

References

[1] T. Kim, X. Wang, N. Zeldovich, M. F. Kaashoek. Intrusion recovery using selective re-execution. *OSDI*, 2010.

[2] E. N. Elnozahy, L. Alvisi, Y.-M. Wang, D. B. Johnson. A survey of rollback-recovery protocols in message-passing systems. *ACM Comput. Surv.*, 34(3), 2002.

[3] H. Garcia-Molina, K. Salem. Sagas. *SIGMOD*, 1987.

[4] P. Buneman, S. Khanna, W.-C. Tan. Why and where: A characterization of data provenance. *ICDT*, 2001.

[5] K. Greshake et al. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. *AISec*, 2023.

[6] E. Debenedetti et al. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. *NeurIPS Datasets and Benchmarks*, 2024.

[7] M. Cemri et al. Why do multi-agent LLM systems fail? arXiv:2503.13657, 2025.

[8] REVISE. arXiv:2609.00643, 2026.

[9] Dependency-guided rollback repair. arXiv:2608.10502, 2026.

A Timeline and Milestones

Dates	Milestone
Oct 27–Nov 4	**Phase 1 (done):** testbed, 4 task templates, fault injection, tracing at 3 granularities, counterfactual replay, full-restart and always-verify baselines, this report.
Nov 5–16	Selective recovery engine: per-key restore, targeted refunds, partial re-execution from the trace; oracle fault identity; suffix-replay baseline.
Nov 17–26	Precision vs. bookkeeping study: granularity sweep, comparison-aware citations, re-validating re-sampled outputs; budget-matched comparison of all methods.
Nov 27–Dec 3	Larger suite (more seeds/templates, source-level injections, second model); stretch: recovery after other agents changed shared state post-fault.
Dec 4–10	Final experiments, figures, final report (due Dec 10).